Meituan Integration — Technical & Risk Overview

How the Roots system syncs stock to the Meituan merchant platform · Prepared 2026-06-23

1. Executive summary

The Roots platform keeps each store's stock numbers up to date on Meituan (■■■■) automatically, so that when an item runs low or sells out in our system, the same is reflected on Meituan within minutes and customers can't over-order. This is done by a background service that signs in to our own Meituan merchant account and updates stock the same way a staff member would when editing a product by hand.

There is no official Meituan partner API available to us, so the integration drives the merchant back office directly. This works reliably, but it operates against Meituan's automation policy, which carries an account-level risk (a verification challenge or, in the worst case, account suspension). This document explains how it works, the risks, the realistic likelihood of being flagged, and the measures already in place to keep that likelihood low.

2. How it works

A background service (the "sync daemon") runs continuously on our Hong Kong server. It holds a logged-in Meituan merchant session per store and watches an internal queue of stock changes. When a product's stock changes in Roots, a job is added to the queue for that store; the daemon then pushes the new number to the matching Meituan listing.

2.1 Per-store routing

Each queued change is tagged with the store it belongs to (e.g. Tianhe / ■■ or Yuexiu / ■■). The daemon groups changes by store and pushes each store's numbers through that store's own Meituan login. Stores never cross over — a Tianhe change only ever updates the Tianhe Meituan account.

2.2 The stock update itself

To set a product's stock, the service uses Meituan's own internal request — the exact same one the merchant website sends when a staff member types a new number into the stock field. Identifying the product is done in three steps, all using Meituan's own data:

- **Find the product** — type the product name into Meituan's product search; Meituan returns the product's internal id (*spuId*).
- **Find the variant** — open that product's editor; Meituan returns the stock-keeping id (*skuId*) and the current stock.
- **Set the stock** — send one request that writes the new number (with auto-replenish turned off so the value holds).

Internally, the final step is a single web request of the form:

```
POST /reuse/product/food/w/batchUpdateSkuStock
spuIds, skuIds, wmPoiId(store), stock=<N>, autoRefresh=0 → { code: 0, msg: "success" }
```

Because the request reuses our existing logged-in session, it needs nothing beyond being signed in — there is no hacking, no third party, and no access to anyone else's data. It is our own account, our own store, our own stock, performed by software instead of by hand.

3. Key components

Component	Role
Sync daemon	Background service that watches the stock-change queue and pushes updates per store.
Headless browser session	A logged-in Meituan merchant session per store, kept alive and reused.
Regional proxy	Outbound traffic is routed through a Guangzhou server so the connection appears to come from the store's region rather than a Hong Kong data centre.
Stock-change queue	Internal list of pending updates; collapses repeated changes so each product is pushed once with its latest value.

4. The risks

The integration is not illegal — it uses our own account and data, and performs actions we are authorised to do. The risk is **contractual and operational**, not legal:

Risk	What it means	Severity
Against Meituan's automation policy	Meituan's merchant terms generally forbid automated/bot access. Using software to drive the back office breaches that policy.	**Medium**
Verification challenge (■■■■)	Meituan may interrupt the session with a human captcha. Sync pauses for that store until a person passes it. This has happened before and is handled gracefully.	**Low–Med**
Account suspension	Worst case: repeated or aggressive automation could lead Meituan to restrict or suspend the merchant account.	**High (rare)**
Platform change	Because there is no official API, a Meituan website change can break the integration until it is updated. This is a maintenance cost, not a safety risk.	**Low**

5. Likelihood of being detected or flagged

No exact percentage is possible — only Meituan knows their detection thresholds and how aggressively they enforce. The figures below are an honest qualitative estimate for **our specific, low-volume usage**, not a guarantee. It is also important to separate two outcomes:

- **Detected** (Meituan's systems notice automation-like behaviour) — more likely.
- **Penalised** (an actual suspension) — much less likely; the usual response is a verification challenge, not a ban.

Scenario	Rough likelihood	Typical outcome
Occasional verification challenge	**Moderate**	Sync pauses for that store; a person passes the captcha; resumes.
Flagged & actually penalised (low-volume use, kept quiet)	**Low — roughly single digits to low-teens % over months**	Account restriction is the rare worst case.

If volume/frequency is increased or run aggressively	**Rises quickly**	More requests = a stronger bot signal = higher chance.

Bottom line: for low-frequency stock syncs, being actually penalised is **low**, and the common real-world outcome is an occasional captcha rather than a ban. The single biggest factor in our favour is **keeping the volume low**; the biggest factor against us is the **data-centre proxy IP** (see §6).

6. What we did to reduce the risk

Several measures are already in place to make the automation look and behave like a normal human operator:

Measure	Why it helps
Lightweight API calls (this update)	Stock is now set with one quiet request instead of loading and clicking through pages — far fewer requests, a much smaller footprint than the old click-the-screen method.
Regional proxy (Guangzhou)	Makes the connection appear to come from the store's own region instead of a Hong Kong data centre — the main signal behind verification challenges.
Browser fingerprint hardening	Removes the tell-tale signs that a browser is automated (the "this is a bot" flags), presenting a consistent, ordinary Windows browser.
Human-like timing	Randomised pauses between actions so the activity never fires on a robotic, perfectly-regular cadence.
Respect store hours & cool-downs	No edits while the store is closed (a 3 a.m. change is a bot tell); after a verification challenge the store pauses and retries later instead of hammering.
De-duplicated, batched updates	Repeated changes collapse to one push of the latest value, and ids are cached — minimising the number of requests made.
One account, normal actions	Only our own merchant account is used, performing only actions a staff member is allowed to do — no scraping of others' data, no privilege escalation.

7. Recommendations to keep risk low

- **Keep sync frequency modest.** This is the single most effective control — low volume is the strongest protection.
- **Consider a residential proxy** instead of the data-centre one. This addresses the biggest remaining detection signal.
- **Treat verification challenges as normal.** When one appears, a person simply passes the captcha; it is not a sign of a ban.
- **Avoid bursts.** Pushing hundreds of changes in a tight loop is the behaviour most likely to be flagged.
- **Have a manual fallback.** Stock can always be set by hand in the merchant app if the integration is ever paused.

Disclaimer: This document is an internal technical and operational summary. The likelihood figures are informed estimates, not guarantees, and this is not legal advice. Whether to operate the integration, and at what frequency, is a business decision.